

MASTERING MongoDB

A Complete Practical Guide for MERN Backend Development

Concepts | CRUD | Aggregation | Indexing | Replication | Sharding | Deployment

How To Read This Book

This book teaches MongoDB from scratch for MERN stack developers. Every topic follows the same pattern: WHAT the concept is, WHY it exists, HOW to use it with real commands, ALTERNATIVES to consider, and GOTCHAS to avoid.

Open your MongoDB shell (mongosh) and type every command yourself as you read. Understanding comes from doing.

CHAPTER 1 | Overview of MongoDB

1.1 What is MongoDB?

MongoDB is a document-oriented NoSQL database. Instead of rows and columns like MySQL or PostgreSQL, MongoDB stores data as flexible JSON-like documents called BSON (Binary JSON). Each document can have a completely different structure — no fixed schema is required.

Think of it this way: a SQL table is like a spreadsheet where every row must use the same columns. A MongoDB collection is like a folder of sticky notes — each note can have different fields written on it.

SQL vs MongoDB — Side by Side

SQL Concept	MongoDB Equivalent	Notes
Database	Database	Same concept
Table	Collection	No fixed schema required
Row / Record	Document	Stored as BSON (JSON-like)
Column	Field	Each document can have different fields

Primary Key	<code>_id</code> field	Auto-generated ObjectId by default
JOIN	<code>\$lookup</code> aggregation	Or embed related data in same document
Schema	Optional (Mongoose adds it)	Flexible by default

1.2 Advantages of MongoDB

- Flexible Schema: Documents in the same collection can have different fields. Add new fields any time — no ALTER TABLE needed.
- Horizontal Scaling (Sharding): Distribute data across many servers. SQL typically scales vertically (bigger machine).
- JSON-like Documents: Data looks like JavaScript objects — natural for Node.js developers.
- Rich Query Language: Filter, project, sort, aggregate — all in one system.
- Built-in Replication: Replica sets give automatic failover and data redundancy.
- High Write Throughput: Designed for high-volume, high-speed operations.
- Geospatial and Full-Text Search: Built-in support without extra plugins.

1.3 Disadvantages of MongoDB

- No strict ACID across multiple collections by default (multi-document transactions added in v4.0).
- Not ideal for highly relational data with complex JOINS between many tables.
- Higher memory usage: indexes and working data are kept in RAM for speed.
- No schema enforcement by default — can lead to inconsistent data (Mongoose solves this).

1.4 When to Use MongoDB vs SQL

Use MongoDB When...	Use SQL (PostgreSQL/MySQL) When...
Data structure varies between records	Data is highly structured and consistent
You need to scale horizontally fast	You need complex multi-table JOINS
Working with JSON APIs (MERN stack)	Financial or banking systems (strict ACID)
Content management, catalogs, logs	Systems with many foreign key relationships
Real-time analytics, IoT, user activity	Mature BI reporting tools that expect SQL

1.5 Setting Up the Environment

Option A: Install MongoDB Locally

```
# macOS (Homebrew)
brew tap mongodb/brew
```

```
brew install mongodb-community@7.0
brew services start mongodb-community@7.0

# Ubuntu / Debian
wget -qO - https://www.mongodb.org/static/pgp/server-7.0.asc | sudo apt-key add -
echo 'deb [ arch=amd64,arm64 ] https://repo.mongodb.org/apt/ubuntu focal/mongodb-org/7.0 multiverse' | sudo tee /etc/apt/sources.list.d/mongodb-org-7.0.list
sudo apt-get update && sudo apt-get install -y mongodb-org
sudo systemctl start mongod

# Windows: Download the MSI installer from
https://www.mongodb.com/try/download/community
```

Option B: MongoDB Atlas (Free Cloud — Recommended for Beginners)

- Go to <https://cloud.mongodb.com> and create a free account.
- Create a free M0 cluster (512MB storage, always free).
- Add your IP address to the network access list.
- Create a database user with a username and password.
- Copy the connection string — you will use it in Node.js.

Open the MongoDB Shell (mongosh)

```
# Connect to local MongoDB
mongosh

# Connect to Atlas
mongosh 'mongodb+srv://username:password@cluster0.xxxxx.mongodb.net/mydb'

# You will see the prompt:
# test>
```

GUI Tip

Install MongoDB Compass (free official GUI) to browse databases visually. Download from <https://www.mongodb.com/products/compass>

CHAPTER 2 | Data Modeling, Databases & Collections

2.1 How MongoDB Organizes Data

MongoDB has three levels of organization, from biggest to smallest:

- Server: The running MongoDB process. Can host many databases.
- Database: A container for related collections (like one app or project).
- Collection: A group of documents (like a table, but schema-less).
- Document: A single record stored as BSON — looks exactly like a JavaScript object.

```
// A MongoDB document looks exactly like a JavaScript object:
{
  _id: ObjectId('64a7f1b2c3d4e5f6a7b8c9d0'), // auto-generated unique ID
  name: 'Alice',
  email: 'alice@example.com',
  age: 28,
  address: { // embedded (nested) document
    city: 'Mumbai',
    state: 'Maharashtra',
    pin: '400001'
  },
  hobbies: ['reading', 'coding', 'chess'], // array field
  createdAt: ISODate('2024-01-15T10:30:00Z')
}
```

2.2 MongoDB Data Types

Data Type	Example	Notes
String	"Alice"	UTF-8 text
Int32	28	32-bit integer
Int64	NumberLong(9000000000)	Use for very large integers
Double	3.14	64-bit floating point
Decimal128	NumberDecimal('9.99')	Precise decimals — use for money
Boolean	true / false	
Date	ISODate('2024-01-15')	Always store as Date, not string
ObjectId	ObjectId('64a7...')	12-byte unique ID, default for _id
Array	['a','b','c']	Ordered list of any values
Object	{ city: 'Mumbai' }	Nested document
Null	null	Absence of value

2.3 Working with Databases

```
// Show all existing databases
show dbs

// Switch to a database (creates it automatically on first insert)
use shopdb

// Check the current database
db

// Drop (permanently delete) the current database and all its data
db.dropDatabase()
```

2.4 Working with Collections

```
// Show all collections in the current database
show collections

// Create a collection explicitly
// (Usually unnecessary - MongoDB auto-creates on first insert)
db.createCollection('users')

// Create a capped collection (fixed-size, oldest docs auto-deleted)
db.createCollection('logs', {
  capped: true,
  size: 1048576, // max 1 MB
  max: 1000      // max 1000 documents
})

// Rename a collection
db.users.renameCollection('customers')

// Drop (delete) a collection and all its documents
db.users.drop()

// Get statistics about a collection
db.users.stats()
```

2.5 Data Modeling: Embedding vs Referencing

Strategy 1 — Embedding (Denormalization)

Store related data inside the same document. Best when you always access the data together and the nested data is owned by the parent document (one-to-one or one-to-few relationships).

```
// Blog post with comments EMBEDDED inside it
{
  _id: ObjectId('post1'),
```

```
title: 'Learning MongoDB',
author: 'Alice',
comments: [
  { user: 'Bob', text: 'Great post!', date: ISODate('2024-01-16') },
  { user: 'Carol', text: 'Very helpful', date: ISODate('2024-01-17') }
]
}

// PROS: One query gets everything. No JOINS needed.
// CONS: Document grows with every comment. 16MB document size limit.
// USE WHEN: Data is few, always read together, owned by the parent.
```

Strategy 2 — Referencing (Normalization)

Store related data in separate collections and link them by `_id`. Best for one-to-many or many-to-many where the related data is large or accessed independently.

```
// Orders and users in SEPARATE collections

// users collection:
{ _id: ObjectId('user1'), name: 'Alice', email: 'alice@example.com' }

// orders collection (each order stores userId as a reference):
{ _id: ObjectId('order1'), userId: ObjectId('user1'), total: 5999, status: 'shipped' }
{ _id: ObjectId('order2'), userId: ObjectId('user1'), total: 1299, status: 'pending' }

// Find all orders for Alice:
db.orders.find({ userId: ObjectId('user1') })

// PROS: No data duplication. Related data can be updated independently.
// CONS: Need $lookup (aggregation join) to combine data.
// USE WHEN: Many documents share the same related data (e.g., many orders, one user).
```

Modeling Rule

Embed when you say 'contains' (a post CONTAINS comments). Reference when you say 'belongs to' (an order BELONGS TO a user). Start with embedding and refactor only if documents get too large.

CHAPTER 3 | CRUD — Insert, Query, Update & Delete

3.1 INSERT — Adding Documents

insertOne() — Insert a single document

```
use shopdb

// insertOne() adds ONE document to the 'products' collection
// MongoDB creates the collection automatically if it doesn't exist
db.products.insertOne({
  name: 'Mechanical Keyboard',
  brand: 'Keychron',
  price: 4999,
  category: 'electronics',
  inStock: true,
  tags: ['keyboard', 'mechanical', 'rgb'],
  specs: { keys: 87, switchType: 'red', wireless: true }
})

// OUTPUT:
// { acknowledged: true, insertedId: ObjectId('64a7...') }
// MongoDB auto-generates the _id using ObjectId
```

insertMany() — Insert multiple documents at once

```
// insertMany() — pass an ARRAY of documents
// Much faster than calling insertOne() in a loop
db.products.insertMany([
  {
    name: 'Wireless Mouse',
    brand: 'Logitech',
    price: 1999,
    category: 'electronics',
    inStock: true,
    tags: ['mouse', 'wireless'],
    specs: { dpi: 1600, buttons: 6 }
  },
  {
    name: 'USB Hub',
    brand: 'Anker',
    price: 899,
    category: 'accessories',
    inStock: false,
    tags: ['usb', 'hub'],
    specs: { ports: 7, type: 'USB-C' }
  },
  {
    name: 'Monitor Stand',
    brand: 'Ergotron',
    price: 6499,
    category: 'accessories',
    inStock: true,
  }
])
```

```
    tags: ['stand', 'ergonomic']
  }
})

// By default: if one insert fails, the rest STOP (ordered: true)
// To insert all and skip failures:
db.products.insertMany([...], { ordered: false })
```

3.2 QUERY — Finding Documents

find() — The primary way to read data

```
// Find ALL documents
db.products.find()

// Filter by exact match (like WHERE in SQL)
db.products.find({ category: 'electronics' })

// Multiple conditions — all must match (implicit AND)
db.products.find({ category: 'electronics', inStock: true })

// Projection — choose which fields to show (1=include, 0=exclude)
db.products.find(
  { category: 'electronics' },           // filter
  { name: 1, price: 1, _id: 0 }         // show name and price, hide _id
)

// findOne() — returns only the FIRST matching document
db.products.findOne({ name: 'Wireless Mouse' })
```

Comparison Operators

```
// $eq — equal (same as writing value directly)
db.products.find({ price: { $eq: 1999 } })

// $ne — not equal
db.products.find({ category: { $ne: 'electronics' } })

// $gt — greater than
db.products.find({ price: { $gt: 1000 } })

// $gte — greater than or equal
db.products.find({ price: { $gte: 1000 } })

// $lt — less than
db.products.find({ price: { $lt: 5000 } })

// $lte — less than or equal
db.products.find({ price: { $lte: 5000 } })
```



```
// Combine: price between 1000 and 5000
db.products.find({ price: { $gte: 1000, $lte: 5000 } })

// $in - value must be IN this list
db.products.find({ category: { $in: ['electronics', 'accessories'] } })

// $nin - value must NOT be in this list
db.products.find({ category: { $nin: ['furniture', 'clothing'] } })
```

Logical Operators

```
// $and - all conditions must be true
db.products.find({
  $and: [
    { price: { $gte: 1000 } },
    { inStock: true }
  ]
})
// Shorthand (implicit AND - same result):
db.products.find({ price: { $gte: 1000 }, inStock: true })

// $or - at least one condition must be true
db.products.find({
  $or: [
    { price: { $lt: 1000 } },
    { category: 'accessories' }
  ]
})

// $not - negates a condition
db.products.find({ price: { $not: { $gt: 5000 } } })

// $nor - none of the conditions must be true
db.products.find({
  $nor: [
    { price: { $gt: 6000 } },
    { inStock: false }
  ]
})
```

Element & Array Operators

```
// $exists - check if a field exists in the document
db.products.find({ specs: { $exists: true } })
db.products.find({ discount: { $exists: false } })

// $type - filter by BSON data type
db.products.find({ price: { $type: 'double' } })

// Array queries - MongoDB checks INSIDE arrays automatically
```

```
// Find documents where tags array CONTAINS 'wireless'
db.products.find({ tags: 'wireless' })

// $all - array must contain ALL of these values
db.products.find({ tags: { $all: ['keyboard', 'rgb'] } })

// $size - array has exactly this many elements
db.products.find({ tags: { $size: 3 } })

// $elemMatch - at least one array element matches ALL conditions
// Useful when array contains objects
db.orders.find({
  items: { $elemMatch: { product: 'Keyboard', qty: { $gte: 2 } } }
})

// Query nested document field using dot notation
db.products.find({ 'specs.wireless': true })
db.products.find({ 'specs.dpi': { $gte: 1600 } })
```

3.3 Sorting, Limiting & Pagination

```
// sort(): 1 = ascending (A to Z, 0 to 9), -1 = descending (Z to A, 9 to 0)
db.products.find().sort({ price: 1 })      // cheapest first
db.products.find().sort({ price: -1 })     // most expensive first

// Sort by multiple fields
db.products.find().sort({ category: 1, price: -1 })
// Groups by category A-Z, then within each category sorts by price high-low

// limit(): return only N documents
db.products.find().limit(5)

// skip(): skip the first N documents (for pagination)
db.products.find().skip(10).limit(10)

// PAGINATION PATTERN:
// Page 1: skip(0).limit(10)   documents 1 to 10
// Page 2: skip(10).limit(10)  documents 11 to 20
// Page 3: skip(20).limit(10)  documents 21 to 30

var page = 2;
var pageSize = 10;
db.products.find()
  .sort({ createdAt: -1 })
  .skip((page - 1) * pageSize)
  .limit(pageSize)

// countDocuments() - count how many match
db.products.countDocuments({ inStock: true })

// distinct() - get unique values of a field
```

```
db.products.distinct('category')
// Returns: ['accessories', 'electronics']
```

3.4 UPDATE — Modifying Documents

updateOne() — Update the first matching document

```
// updateOne(filter, updateOperator, options)

// $set - set fields to new values (creates field if missing)
db.products.updateOne(
  { name: 'Wireless Mouse' },
  { $set: { price: 2199, inStock: true } }
)

// $unset - remove a field from the document
db.products.updateOne(
  { name: 'USB Hub' },
  { $unset: { discount: '' } }
)

// $inc - increment a numeric field (use negative to decrement)
db.products.updateOne(
  { name: 'Mechanical Keyboard' },
  { $inc: { price: 200, views: 1 } }
)

// $mul - multiply a field by a factor
db.products.updateOne(
  { name: 'Monitor Stand' },
  { $mul: { price: 1.10 } } // 10% price increase
)

// $rename - rename a field
db.products.updateOne(
  { name: 'Wireless Mouse' },
  { $rename: { 'specs': 'specifications' } }
)

// $min - update only if new value is LESS than current
// $max - update only if new value is MORE than current
db.products.updateOne(
  { name: 'Wireless Mouse' },
  { $min: { price: 1800 } } // sets price to 1800 only if 1800 < current price
)

// upsert: true - insert if not found, update if found
db.products.updateOne(
  { name: 'New Product' },
  { $set: { price: 500, category: 'accessories' } },
  { upsert: true }
)
```

updateMany() — Update ALL matching documents

```
// Apply 10% price increase to all electronics
db.products.updateMany(
  { category: 'electronics' },
  { $mul: { price: 1.10 } }
)

// Add a 'views' field to every document that doesn't have it yet
db.products.updateMany(
  { views: { $exists: false } },
  { $set: { views: 0 } }
)
```

Array Update Operators

```
// $push — add an element to an array
db.products.updateOne(
  { name: 'Mechanical Keyboard' },
  { $push: { tags: 'gaming' } }
)

// $push with $each — add multiple elements
db.products.updateOne(
  { name: 'Mechanical Keyboard' },
  { $push: { tags: { $each: ['productivity', 'tenkeyless'] } } }
)

// $addToSet — add only if it doesn't already exist (no duplicates)
db.products.updateOne(
  { name: 'Wireless Mouse' },
  { $addToSet: { tags: 'wireless' } }
)

// $pull — remove all elements matching the value
db.products.updateOne(
  { name: 'Mechanical Keyboard' },
  { $pull: { tags: 'rgb' } }
)

// $pop — remove first (-1) or last (1) element
db.products.updateOne(
  { name: 'Wireless Mouse' },
  { $pop: { tags: -1 } } // removes the first element
)
```

replaceOne() — Replace the entire document

```
// replaceOne swaps the ENTIRE document (except _id)
// Unlike updateOne which only changes specified fields
```

```
db.products.replaceOne(
  { name: 'USB Hub' },
  {
    name: 'USB Hub',
    brand: 'Anker',
    price: 999,
    category: 'accessories',
    inStock: true
    // ALL old fields (tags, specs) are REMOVED because we replaced the doc
  }
)
```

3.5 DELETE — Removing Documents

```
// deleteOne() - delete the FIRST document matching the filter
db.products.deleteOne({ name: 'USB Hub' })
// Output: { acknowledged: true, deletedCount: 1 }

// deleteMany() - delete ALL documents matching the filter
db.products.deleteMany({ inStock: false })

// Delete ALL documents in a collection (empty filter = match everything)
db.products.deleteMany({})

// findOneAndDelete() - delete AND return the deleted document
var deleted = db.products.findOneAndDelete({ name: 'Monitor Stand' })
// 'deleted' now contains the document that was just removed

// findOneAndUpdate() - update AND return the document
db.products.findOneAndUpdate(
  { name: 'Wireless Mouse' },
  { $inc: { price: 100 } },
  { returnDocument: 'after' } // 'before' or 'after'
)
```

CHAPTER 4 | Indexing — Making Queries Fast

4.1 What is an Index?

Without an index, MongoDB scans EVERY document to find matches. This is called a Collection Scan (COLLSCAN) and is very slow for large collections with thousands or millions of records.

An index is a separate sorted data structure (a B-tree) that holds specific field values with pointers back to the original documents. Queries on indexed fields look up the index first — like using a book index instead of reading every page.

Speed Impact

A query on 1 million documents: WITHOUT index = scan all 1M docs (2-5 seconds). WITH index = jump directly to results (1-5 milliseconds). That is up to 1000x faster.

4.2 Creating and Managing Indexes

```
// View all existing indexes on a collection
db.products.getIndexes()
// Every collection has a default index on _id automatically

// SINGLE FIELD INDEX
// 1 = ascending order, -1 = descending order
db.products.createIndex({ price: 1 })
// Now queries filtering or sorting by price are fast

// COMPOUND INDEX - index on multiple fields together
// Helps queries that filter on 'category' AND sort by 'price'
db.products.createIndex({ category: 1, price: -1 })
// ESR Rule: Equality fields first, Sort fields second, Range fields last

// UNIQUE INDEX - no two documents can have the same value for this field
db.users.createIndex({ email: 1 }, { unique: true })

// SPARSE INDEX - only indexes documents where the field EXISTS
db.products.createIndex({ discount: 1 }, { sparse: true })

// TTL INDEX - automatically deletes documents after a time period
// Perfect for sessions, OTP tokens, expiring data
db.sessions.createIndex(
  { createdAt: 1 },
  { expireAfterSeconds: 3600 } // deletes document 1 hour after createdAt
)

// TEXT INDEX - enables full-text search on string fields
db.products.createIndex({ name: 'text', description: 'text' })

// Drop (delete) an index
db.products.dropIndex({ price: 1 })
db.products.dropIndex('price_1') // using the auto-generated index name
```

```
// Drop ALL indexes (except _id)
db.products.dropIndexes()
```

4.3 Analyzing Performance with explain()

```
// explain() shows HOW MongoDB executed a query
// Always run this to verify your indexes are working!
db.products.find({ price: { $gte: 1000 } }).explain('executionStats')

// KEY THINGS TO LOOK FOR in the output:
//
// 'stage': 'COLLSCAN' = BAD: scanned every document, no index used
// 'stage': 'IXSCAN'   = GOOD: used an index
//
// 'nReturned': 3           = matched 3 documents
// 'totalDocsExamined': 1000 = scanned 1000 docs to find 3 = needs index!
// 'executionTimeMillis': 450 = took 450ms – too slow

// AFTER adding an index:
db.products.createIndex({ price: 1 })
db.products.find({ price: { $gte: 1000 } }).explain('executionStats')
// Now you should see:
// 'stage': 'IXSCAN'
// 'totalDocsExamined': 3   = only examined exactly the matching 3 docs
// 'executionTimeMillis': 1 = 1ms!
```

Index Trade-off

Indexes speed up reads but slow down writes because MongoDB must update the index on every insert, update, and delete. Only create indexes on fields you frequently query or sort by. Too many indexes = slow writes and wasted RAM.

CHAPTER 5 | Aggregation Framework

5.1 What is Aggregation?

Aggregation processes multiple documents and computes results — like totals, averages, counts, and complex transformations. It is far more powerful than `find()` for analytics and reporting.

The aggregation pipeline is a sequence of stages. Documents flow through each stage like an assembly line. Each stage transforms the data and passes results to the next stage.

```
// Basic aggregation structure:
db.collection.aggregate([
  { $stage1: { ... } }, // stage 1: all docs pass through
  { $stage2: { ... } }, // stage 2: gets output of stage 1
  { $stage3: { ... } }, // stage 3: gets output of stage 2
])
```

5.2 Essential Aggregation Stages

\$match — Filter documents (like `find()`)

```
// Always put $match FIRST for best performance
// It reduces the number of docs before expensive stages run
db.products.aggregate([
  { $match: { category: 'electronics', inStock: true } }
])
```

\$group — Group and compute aggregates

```
// _id defines what to GROUP BY
// null in _id = group ALL documents into one group
db.products.aggregate([
  {
    $group: {
      _id: '$category', // GROUP BY category
      totalProducts: { $sum: 1 }, // count docs in each group
      avgPrice: { $avg: '$price' }, // average price
      minPrice: { $min: '$price' }, // cheapest in group
      maxPrice: { $max: '$price' }, // most expensive
      totalValue: { $sum: '$price' }, // sum of all prices
      names: { $push: '$name' } // collect names into array
    }
  }
])

// GROUP operators reference:
// $sum: 1 count documents
// $sum: '$field' sum field values
// $avg: '$field' average
// $min / $max minimum / maximum value
// $push: '$field' collect all values into array
```



```
// $addToSet      collect UNIQUE values into array
// $first / $last  first or last value in the group
```

\$project — Shape the output fields

```
db.products.aggregate([
  {
    $project: {
      _id: 0,          // 0 = exclude
      name: 1,         // 1 = include
      price: 1,
      // Computed fields:
      priceWithTax: { $multiply: ['$price', 1.18] },
      nameUpper: { $toUpper: '$name' },
      label: { $concat: ['$name', ' by ', '$brand'] },
    }
  }
])
```

\$sort, \$limit, \$skip — Order and page results

```
db.products.aggregate([
  { $match: { inStock: true } },
  { $sort: { price: -1 } }, // descending
  { $skip: 0 },
  { $limit: 5 }
])
```

\$lookup — Join two collections (like SQL JOIN)

```
// Join orders with the users collection
db.orders.aggregate([
  {
    $lookup: {
      from: 'users',          // other collection
      localField: 'userId',    // field in orders
      foreignField: '_id',     // field in users
      as: 'userDetails'       // name for the new array field
    }
  },
  // $lookup produces an array — $unwind converts it to a single object
  { $unwind: '$userDetails' }
])

// Result: each order now has the full user object embedded:
// { _id: ObjectId('order1'), total: 5999,
//   userDetails: { _id: ObjectId('user1'), name: 'Alice', email: '...' } }
```

\$unwind — Deconstruct an array into separate documents

```
// Before $unwind:
// { name: 'Keyboard', tags: ['keyboard','mechanical','rgb'] }

// After $unwind on 'tags':
// { name: 'Keyboard', tags: 'keyboard' }
// { name: 'Keyboard', tags: 'mechanical' }
// { name: 'Keyboard', tags: 'rgb' }

// Real use: find the most common tags across all products
db.products.aggregate([
  { $unwind: '$tags' },
  { $group: { _id: '$tags', count: { $sum: 1 } } },
  { $sort: { count: -1 } }
])
```

\$addFields — Add computed fields without losing existing ones

```
db.products.aggregate([
  {
    $addFields: {
      discountedPrice: { $multiply: ['$price', 0.9] },
      isExpensive:      { $gt: ['$price', 3000] },
      tagCount:         { $size: '$tags' }
    }
  }
])
```

5.3 Real-World Aggregation Example

```
// 'Show each category: total value, average price, product count'
// 'Only categories with more than 1 product, sorted by total value.'

db.products.aggregate([
  // Stage 1: Only in-stock products
  { $match: { inStock: true } },

  // Stage 2: Group by category
  {
    $group: {
      _id: '$category',
      productCount: { $sum: 1 },
      totalValue:    { $sum: '$price' },
      avgPrice:      { $avg: '$price' },
      productNames: { $push: '$name' }
    }
  },

  // Stage 3: Filter groups with more than 1 product
  { $match: { productCount: { $gt: 1 } } },
])
```

```
// Stage 4: Sort by total value descending
{ $sort: { totalValue: -1 } },

// Stage 5: Shape the output
{
  $project: {
    category:      '$_id',
    _id:           0,
    productCount: 1,
    totalValue:    1,
    avgPrice:      { $round: ['$avgPrice', 2] },
    productNames: 1
  }
}
])
```

CHAPTER 6 | Relationships, Text Search & Regex

6.1 Relationships in MongoDB

One-to-One — Embedded

```
// User with their profile embedded (always accessed together)
{
  _id: ObjectId('user1'),
  email: 'alice@example.com',
  profile: {
    bio: 'Developer from Mumbai',
    avatar: '/images/alice.jpg',
    website: 'alice.dev'
  }
}
```

One-to-Many — Referenced

```
// Users and Orders - one user has many orders

// users collection:
{ _id: ObjectId('user1'), name: 'Alice', email: 'alice@example.com' }

// orders collection - each order stores userId:
{ _id: ObjectId('order1'), userId: ObjectId('user1'), total: 5999 }
{ _id: ObjectId('order2'), userId: ObjectId('user1'), total: 1299 }

// Find all orders for Alice:
db.orders.find({ userId: ObjectId('user1') })

// Join with $lookup:
db.users.aggregate([
  { $match: { _id: ObjectId('user1') } },
  { $lookup: { from: 'orders', localField: '_id', foreignField: 'userId', as: 'orders' } }
])
```

Many-to-Many — Reference Array

```
// Students and Courses: a student takes many courses,
// a course has many students

// students collection:
{
  _id: ObjectId('student1'),
  name: 'Alice',
  enrolledCourses: [ObjectId('course1'), ObjectId('course2')]
}
```

```
// courses collection:
{ _id: ObjectId('course1'), title: 'MongoDB Mastery', instructor: 'Prof. Smith' }

// Get all course details for Alice:
db.students.aggregate([
  { $match: { _id: ObjectId('student1') } },
  {
    $lookup: {
      from: 'courses',
      localField: 'enrolledCourses',
      foreignField: '_id',
      as: 'courseDetails'
    }
  }
])
```

6.2 Full-Text Search

```
// Step 1: Create a text index (a collection can have only ONE text index)
db.products.createIndex({ name: 'text', description: 'text' })

// Step 2: Search using $text
db.products.find({ $text: { $search: 'keyboard mechanical' } })
// Finds documents containing 'keyboard' OR 'mechanical' in indexed fields

// Exact phrase search (wrap in escaped quotes)
db.products.find({ $text: { $search: '"mechanical keyboard"' } })

// Exclude a word (prefix with -)
db.products.find({ $text: { $search: 'keyboard -wireless' } })

// Sort by relevance score (best match first)
db.products.find(
  { $text: { $search: 'keyboard' } },
  { score: { $meta: 'textScore' } }
).sort({ score: { $meta: 'textScore' } })

// Text search is case-insensitive and ignores stop words (the, is, a, etc.)
```

6.3 Regular Expressions

```
// Basic regex search - case-insensitive match
db.products.find({ name: /key/i })

// Using $regex operator
db.products.find({ name: { $regex: 'key', $options: 'i' } })

// $options flags:
// 'i' = case insensitive
```

```
// 'm' = multiline (^ and $ match line boundaries)
// 's' = allows dot to match newline characters

// Starts with 'Wire' (case-insensitive)
db.products.find({ name: /^Wire/i })

// Ends with 'Hub'
db.products.find({ name: /Hub$/i })

// Contains a number
db.products.find({ name: /[0-9]/ })

// Find gmail users
db.users.find({ email: /@gmail\.com$/i })

// NOTE: Regex is slow without an index!
// Only regex anchored to the START (^) can use an index.
// For full-text search, always prefer text indexes.
```

CHAPTER 7 | Replication & Sharding

7.1 Replication — High Availability

Replication keeps multiple copies of the same data on different servers. Benefits: High Availability (automatic failover if a server crashes) and Data Redundancy (data is safe even if a disk fails).

MongoDB replication is built around the Replica Set — a group of MongoDB servers that all hold identical data.

Member Type	Role
Primary	The only node that accepts write operations. One primary per replica set.
Secondary	Copies data from the primary continuously. Can serve reads if configured.
Arbiter	Lightweight member (no data). Participates in elections to break ties.

```
// Initialize a replica set (connect to one mongod instance first)
rs.initiate({
  _id: 'rs0',
  members: [
    { _id: 0, host: 'localhost:27017' }, // becomes primary
    { _id: 1, host: 'localhost:27018' }, // secondary
    { _id: 2, host: 'localhost:27019' }, // secondary
  ]
})

// Check replica set status
rs.status()
// Look for 'stateStr': 'PRIMARY' or 'SECONDARY'

// Force the current primary to step down (triggers new election)
rs.stepDown()

// Add a new member
rs.add('localhost:27020')

// Add an arbiter
rs.addArb('localhost:27021')
```

Write Concerns & Read Preferences

```
// Write Concern — how many nodes must confirm the write before success
// w: 1          only primary confirms (fast, slight risk)
// w: 'majority' more than half confirm (safer, slightly slower)
db.products.insertOne(
  { name: 'New Item', price: 999 },
  { writeConcern: { w: 'majority', wtimeout: 5000 } }
```

```
)

// Read Preference - which node to read from
// 'primary'           always read from primary (freshest data)
// 'primaryPreferred'  prefer primary, use secondary if primary down
// 'secondary'         always read from secondary (may be slightly behind)
// 'secondaryPreferred' prefer secondary, fallback to primary
// 'nearest'           read from lowest latency node

// Set in connection string:
// mongodb://host/?replicaSet=rs0&readPreference=secondaryPreferred
```

7.2 Sharding — Horizontal Scaling

Sharding distributes data across multiple machines (shards) when a single server cannot hold all your data. Instead of upgrading to a bigger server, you add more servers and MongoDB splits the data automatically.

Component	Role
Shard	A replica set holding a subset of the total data
mongos	Query router — clients connect here, it routes to the right shard
Config Servers	Store metadata about which data lives on which shard
Shard Key	The field used to decide which shard a document goes to

```
// Enable sharding on a database
sh.enableSharding('shopdb')

// Create index on the shard key first (required)
db.orders.createIndex({ customerId: 1 })

// Shard the orders collection using customerId as the shard key
sh.shardCollection('shopdb.orders', { customerId: 1 })

// Hashed sharding - distributes data evenly, prevents hotspots
sh.shardCollection('shopdb.orders', { customerId: 'hashed' })

// Check sharding status
sh.status()
```

Production Note

In real projects you rarely configure sharding manually. MongoDB Atlas handles it automatically with Multi-Region Clusters. The most important decision is choosing your shard key — a bad shard key causes hotspots and kills performance.

CHAPTER 8 | Map Reduce

8.1 What is Map Reduce?

Map Reduce is a data processing pattern that works in two phases. The Map phase runs on each document and emits key-value pairs. The Reduce phase combines all values for each key into a final result.

IMPORTANT: In MongoDB v5.0+, Map Reduce is deprecated in favor of the Aggregation Framework, which is faster, more readable, and more powerful. You will see Map Reduce only in legacy codebases.

Use Aggregation Instead

For all new code, use `$group`, `$match`, and `$project` in the aggregation pipeline. Map Reduce is 5-10x slower and much harder to debug. It is shown here only for completeness.

```
// Count products per category using Map Reduce

// MAP - runs on each document, emits key-value pairs
var mapFn = function() {
  emit(this.category, 1); // key = category, value = 1
};

// REDUCE - combines values for the same key
var reduceFn = function(key, values) {
  return Array.sum(values); // sums all the 1s
};

// Run Map Reduce
db.products.mapReduce(mapFn, reduceFn, {
  out: 'category_counts', // write results to this collection
  query: { inStock: true } // optional filter
})

db.category_counts.find()
// { _id: 'accessories', value: 2 }
// { _id: 'electronics', value: 2 }

// EQUIVALENT with Aggregation (ALWAYS prefer this):
db.products.aggregate([
  { $match: { inStock: true } },
  { $group: { _id: '$category', count: { $sum: 1 } } }
])
```

CHAPTER 9 | Backup & Deployment

9.1 Backup — mongodump & mongorestore

```
# BACKUP — create a dump of your data

# Dump ALL databases
mongodump --out /backup/dump/

# Dump a specific database
mongodump --db shopdb --out /backup/shopdb/

# Dump a specific collection
mongodump --db shopdb --collection products --out /backup/

# Dump from Atlas (cloud)
mongodump --uri 'mongodb+srv://user:pass@cluster0.xxx.mongodb.net' --out /backup/

# Compress to save space
mongodump --db shopdb --gzip --out /backup/compressed/

# RESTORE — put data back

# Restore everything
mongorestore /backup/dump/

# Restore specific database
mongorestore --db shopdb /backup/shopdb/shopdb/

# Drop existing data first, then restore (clean restore)
mongorestore --drop /backup/dump/
```

mongoexport & mongoimport — JSON and CSV

```
# Export collection to JSON
mongoexport --db shopdb --collection products --out products.json

# Export to CSV with specific fields
mongoexport --db shopdb --collection products --type csv --fields
name,price,category --out products.csv

# Import from JSON
mongoimport --db shopdb --collection products --file products.json

# Import from CSV
mongoimport --db shopdb --collection products --type csv --headerline --file
products.csv
```

9.2 Connecting MongoDB to Node.js with Mongoose

Mongoose is the standard ODM (Object Document Mapper) for connecting MongoDB to a Node.js/Express app. It adds schema definitions, validation, and a clean query API on top of the raw MongoDB driver.

```
npm install mongoose
```

```
// db.js - database connection
const mongoose = require('mongoose');

const connectDB = async () => {
  try {
    await mongoose.connect(process.env.MONGODB_URI);
    console.log('MongoDB connected successfully');
  } catch (err) {
    console.error('MongoDB connection failed:', err.message);
    process.exit(1);
  }
};

module.exports = connectDB;
```

```
// models/Product.js - Define a Mongoose Schema
const mongoose = require('mongoose');

const productSchema = new mongoose.Schema({
  name: {
    type: String,
    required: [true, 'Product name is required'],
    trim: true,
    maxlength: [100, 'Name cannot exceed 100 characters']
  },
  price: {
    type: Number,
    required: true,
    min: [0, 'Price cannot be negative']
  },
  category: {
    type: String,
    enum: ['electronics', 'accessories', 'furniture'],
    required: true
  },
  inStock: { type: Boolean, default: true },
  tags: [String],
}, { timestamps: true }); // auto-adds createdAt and updatedAt

// Index at schema level
productSchema.index({ price: 1 });

module.exports = mongoose.model('Product', productSchema);
// 'Product' model → 'products' collection (auto lowercased + pluralized)
```

```
// Using the Mongoose model in Express routes
const Product = require('./models/Product');

// CREATE
const product = await Product.create({ name: 'Keyboard', price: 4999, category:
'electronics' });

// READ
const all      = await Product.find();
const elec     = await Product.find({ category: 'electronics' }).sort({ price: 1
});
const one      = await Product.findById(req.params.id);
const byName   = await Product.findOne({ name: 'Keyboard' });

// UPDATE
const updated = await Product.findByIdAndUpdate(
  req.params.id,
  { $set: { price: 5499 } },
  { new: true, runValidators: true } // new:true returns the updated document
);

// DELETE
await Product.findByIdAndDelete(req.params.id);

// AGGREGATE — works exactly like the MongoDB shell
const stats = await Product.aggregate([
  { $group: { _id: '$category', avgPrice: { $avg: '$price' } } }
]);
```

9.3 Production Checklist

Task	Why It Matters
Use MongoDB Atlas or managed DB	Automatic backups, scaling, security patches
Enable authentication	Always set username and password — never run with auth off
Use TLS/SSL connections	Encrypts data in transit — Atlas does this by default
Whitelist server IP only	Only your app server can connect to the database
Store credentials in .env file	Never put passwords or connection strings in code
Enable automated daily backups	Atlas: enable point-in-time recovery
Index all frequently queried fields	Use explain() to find slow queries before launch
Monitor with Atlas or Datadog	Watch CPU, memory, connections, slow queries
Set appropriate write concern	Use w: 'majority' for critical data writes
Validate schemas with Mongoose	Prevent inconsistent or garbage data from entering

```
# .env file (NEVER commit to Git — add to .gitignore)
```

```
MONGODB_URI=mongodb+srv://myuser:mypassword@cluster0.xxxxx.mongodb.net/shopdb?retryWrites=true&w=majority

// Production Mongoose connection settings
await mongoose.connect(process.env.MONGODB_URI, {
  maxPoolSize: 10,           // max 10 concurrent connections
  serverSelectionTimeoutMS: 5000, // fail fast if cannot connect
  socketTimeoutMS: 45000,    // close idle sockets after 45s
});

// Handle unexpected disconnections
mongoose.connection.on('error', err => console.error('DB error:', err));
mongoose.connection.on('disconnected', () => console.warn('DB disconnected'));

// Clean up on app shutdown
process.on('SIGINT', async () => {
  await mongoose.connection.close();
  process.exit(0);
});
```

Congratulations!

You have completed the full MongoDB guide! You now know how to design schemas, perform all CRUD operations, write complex aggregation pipelines, use indexes effectively, and deploy to production. Combine this with your Express.js knowledge to build a complete, production-ready MERN backend.